

1. What is Regularization?

- Regularization is a technique used in **Deep Learning and Machine Learning** to **prevent overfitting**.
- **Overfitting**: When a model performs well on training data but poorly on new data.
- **Regularization helps the model generalize better** to unseen data.
- Example:
If a student memorizes answers instead of understanding concepts, they may fail in a new exam. Regularization prevents this type of memorization in models.

2. Parameter Norm Penalties

- In deep learning models, we try to **minimize a loss function** so that the predictions are close to the actual values. During training, the model learns many **parameters (weights)**. Sometimes these weights become **very large**, which makes the model memorize the training data instead of learning general patterns. This leads to **overfitting**.
- To avoid this problem, we use **Parameter Norm Penalties**.

What are Parameter Norm Penalties?

- Parameter norm penalties are a **regularization technique** where we **add an extra penalty term to the loss function**. This penalty discourages the model from having very large weight values.
- So instead of minimizing only the original loss, the model minimizes:
- **Total Loss = Original Loss + Regularization Penalty**

- The penalty depends on the **norm (size) of the weight parameters**.
- Purpose:
- Control the size of weights
- Prevent overfitting
- Improve generalization on new data

Types of Parameter Norm Penalties

- There are two common types:
- **L2 Regularization (Weight Decay)**
- **L1 Regularization**

1. L2 Regularization (Weight Decay)

- **Basic Idea**

- L2 regularization adds a penalty equal to the **square of the weight values**.
- The loss function becomes:
- **Loss = Original Loss + $\lambda \sum w^2$**
- Where:
- **Loss** = Total loss used for training
- **Original Loss** = Normal loss (MSE, cross-entropy, etc.)
- **w** = Model weights
- **$\sum w^2$** = Sum of squares of all weights
- **λ (lambda)** = Regularization parameter that controls penalty strength

Why Square of Weights?

- Squaring makes **large weights much more expensive**.
- You can see that **larger weights produce very large penalties**.
- So the model naturally tries to **keep weights small**.

Weight	Square
0.5	0.25
2	4
5	25

- **Example**

Suppose:

Original Loss = 0.8

Weights = 2, 3

Step 1: Square the weights

$$2^2 = 4$$

$$3^2 = 9$$

Step 2: Add them

$$4 + 9 = 13$$

Step 3: Multiply by λ

If $\lambda = 0.01$

$$\text{Penalty} = 0.01 \times 13 = 0.13$$

Step 4: Total Loss

$$\text{Total Loss} = 0.8 + 0.13 = \mathbf{0.93}$$

So the model tries to reduce both:

prediction error

weight size

- **Why L2 is called Weight Decay**
- During training, L2 regularization **gradually reduces the size of weights**.
- Large weights shrink slightly after every update step.
- That is why it is called **weight decay**.

Advantages of L2 Regularization

- **Prevents very large weights**
 - Large weights can make the model unstable. L2 keeps them small.
- **Improves generalization**
 - The model learns **simpler patterns** that work on new data.
- **Reduces overfitting**
 - The model cannot memorize the training data.
- **Keeps all features**
 - Unlike L1, it does not remove features completely.

Visual Intuition

- Without regularization:
- Weights may grow very large to perfectly fit training data.
- With L2 regularization:
- Weights are **smoothly reduced**, making the model simpler.

2. L1 Regularization

- **Basic Idea**
- L1 regularization adds a penalty equal to the **absolute value of weights**.
- The loss function becomes:
- **Loss = Original Loss + $\lambda \sum |w|$**
- Where:
- **$|w|$** = absolute value of weight
- λ = regularization strength

Example

Weights = 2, 3

Step 1: Take absolute values

$$|2| = 2$$

$$|3| = 3$$

Step 2: Add them

$$2 + 3 = 5$$

Step 3: Multiply by λ

If $\lambda = 0.01$

$$\text{Penalty} = 0.01 \times 5 = 0.05$$

Step 4: Total Loss

$$\text{Total Loss} = 0.8 + 0.05 = \mathbf{0.85}$$

- **Example**

- Weights = 2, 3

- Step 1: Take absolute values

- $|2| = 2$

- $|3| = 3$

- Step 2: Add them

- $2 + 3 = 5$

- Step 3: Multiply by λ

- If $\lambda = 0.01$

- $\text{Penalty} = 0.01 \times 5 = 0.05$

- Step 4: Total Loss

- $\text{Total Loss} = 0.8 + 0.05 = \mathbf{0.85}$

Key Property of L1 Regularization

- L1 regularization pushes **some weights exactly to zero**.
- Example:
- Weights before training:
 - $w_1 = 0.5$
 - $w_2 = 0.02$
 - $w_3 = 1.2$
- After L1 regularization:
 - $w_1 = 0.4$
 - $w_2 = \mathbf{0}$
 - $w_3 = 1.0$
- So some features are **removed from the model**.

What is a Sparse Model?

- A **sparse model** means most weights are **zero**.
- Example:
 - $[0, 0, 1.5, 0, 0.8, 0]$
- Only a few weights are active.
- Advantages:
 - Simpler model
 - Faster computation
 - Easier interpretation

Advantages of L1 Regularization

- **Produces sparse models**
 - Many weights become zero.
- **Feature selection**
 - Unimportant features automatically disappear.
- **Simpler models**
 - Only important features remain.

Difference Between L1 and L2 Regularization

- Difference Between L1 and L2 Regularization

Feature	L1 Regularization	L2 Regularization
Penalty	Absolute value	Square of weights
Formula	$\lambda \sum$	w
Weight behavior	Some weights become zero	Weights become small
Model type	Sparse model	Dense model
Feature selection	Yes	No

Simple Intuition

- Think of a teacher grading students.
- Without regularization:
- Students may write **very long answers** just to get marks.
- With regularization:
- Teacher says:
- **L2**: Write clearly but do not exaggerate.
- **L1**: Write only important points.

3. Norm Penalties as Constrained Optimization

Instead of adding penalty in the loss function, we can **limit the value of weights**.

Example:

$$||w|| < c$$

Meaning:

Weights cannot exceed a certain value.

Purpose:

Prevent extremely large parameters

Improve model stability

4. Regularization and Under-Constrained Problems

- Sometimes a model has **many parameters but little data**.
- Example:
 - Deep neural networks with millions of weights
 - Small training dataset
 - This is called an **under-constrained problem**.
 - Regularization helps by:
 1. Restricting parameter values
 2. Avoiding meaningless solutions

5. Dataset Augmentation (Data Augmentation)

- In **deep learning**, models require a **large amount of training data** to learn patterns effectively. However, in many real-world situations, collecting a large dataset is **expensive, time-consuming, or sometimes impossible**.
- To solve this problem, we use **Dataset Augmentation (or Data Augmentation)**.
- **Dataset augmentation** is a technique in which we **artificially increase the size of the training dataset by creating modified versions of existing data**

- Instead of collecting new data, we **transform the existing data in different ways** while keeping the same label.
- Example:
If we have **100 images of cats**, we can generate **many more images** by rotating, flipping, or adjusting brightness.
- So the dataset becomes much larger and more diverse.

Why Dataset Augmentation is Needed

- Deep learning models often contain **millions of parameters**. If the dataset is small, the model may **memorize the training data instead of learning general patterns**. This leads to **overfitting**.
- Dataset augmentation helps by:
- Increasing the **size of training data**
- Increasing **data diversity**
- Helping the model learn **robust features**

Basic Idea of Dataset Augmentation

- Instead of using only the original data:
- **Original Dataset**
 - Image 1
 - Image 2
 - Image 3
- We create **new versions of these images** using transformations.
- Example:
 - Original Image → Rotated Image → Flipped Image → Cropped Image → Brightness Adjusted Image
- Now the model sees **many variations of the same object**.

Common Dataset Augmentation Techniques

- **1. Image Rotation**

- Rotation means **turning the image by a certain angle**.
- Example angles:
 - 10°
 - 30°
 - 90°
- Example:
 - Original image → rotated left → rotated right
- Purpose:
The object can appear in **different orientations in real life**, so the model learns to recognize it from different angles.
- Example:
A cat is still a cat even if the image is slightly rotated.

2. Image Flipping

- Flipping means **mirroring the image**.
- Types of flipping:
 - **Horizontal Flip**
 - Left and right sides are swapped.
 - Example:
Face looking left → face looking right
 - **Vertical Flip**
 - Top and bottom sides are swapped.
 - Example:
Object upside down.
 - Purpose:
Helps the model recognize objects from **different directions**.

3. Cropping

- Cropping means **cutting a portion of the image**.
- Example:
- Original Image → small section of the image
- Purpose:
Helps the model focus on **important parts of the object**.
- Example:
If a dog occupies only part of the image, cropping helps the model learn **local features**.

4. Changing Brightness

- Brightness adjustment means **increasing or decreasing the light level of the image.**
- Example:
 - Original image
 - Brighter image
 - Darker image
- Purpose:
In real life, images may be captured in:
 - sunlight
 - low light
 - shadow
- Brightness augmentation helps the model **work well in different lighting conditions.**

Example of Dataset Augmentation Process

- Suppose we have **one image**.
- Original Image
- Now apply different transformations:
- Rotate the image
- Flip the image
- Crop the image
- Adjust brightness
- Result:
- Original Image
↓
Rotated Image
↓
Flipped Image
↓
Cropped Image
↓
Brightness Adjusted Image
- From **1 image we can generate many new images**

6. Noise Robustness

- Noise robustness means the **model should work even if input data contains noise.**

- Example noise:

1. Blurred images
2. Distorted signals
3. Sensor errors

- Method:

- Add small noise to training data

- Example:

- $x \rightarrow x + \text{noise}$

- Benefit:

1. Model becomes more stable
2. Learns stronger patterns

7. Semi-Supervised Learning

- Uses **both labeled and unlabeled data**.
- Example:
- Dataset:
 1. 100 labeled images
 2. 1000 unlabeled images
- Instead of ignoring unlabeled data, the model learns from both.
- Advantages:
 1. Improves accuracy
 2. Reduces need for large labeled datasets

8. Multi-Task Learning

- A single model learns **multiple tasks simultaneously**.
- Example:
 - One model predicts:
 1. Age
 2. Gender
 3. Emotion
 - Benefits:
 1. Shared knowledge between tasks
 2. Better feature learning
 3. Reduced overfitting

9. Early Stopping

- **Introduction**

- In **deep learning and machine learning**, models learn by training on data for several **iterations called epochs**. During training, the model tries to **reduce the error (loss)** so that its predictions become more accurate.
- However, if the model is trained for **too many epochs**, it may start **memorizing the training data** instead of learning general patterns. This problem is called **overfitting**.
- To avoid this problem, we use a technique called **Early Stopping**.
- **Early Stopping means stopping the training process at the right time before the model starts overfitting.**

9. Early Stopping (Regularization Technique)

- In deep learning and machine learning, a model learns by **training on a dataset for many iterations called epochs**. During training, the model tries to **reduce the error (loss)** so that its predictions become more accurate.
- However, if the model is trained **for too long**, it may start to **memorize the training data** instead of learning general patterns. This problem is called **overfitting**.
- To prevent this, we use a technique called **Early Stopping**.
- **Early stopping means stopping the training process before the model starts overfitting.**

Understanding Dataset Splitting

- Before training a model, the dataset is usually divided into three parts:
- **1. Training Dataset**
 - Used to **train the model**.
 - The model learns patterns from this data.
- **2. Validation Dataset**
 - Used to **check the model performance during training**.
 - Helps us decide **when to stop training**.
- **3. Test Dataset**
 - Used **after training is completed**.
 - Used to measure **final performance** of the model.
 - Early stopping mainly uses the **validation dataset**.

What Happens During Training

- When a model is trained for many epochs, two types of errors are observed:
- **1. Training Error**
 - Error calculated using the **training dataset**.
 - Usually **keeps decreasing** as training continues.
- **2. Validation Error**
 - Error calculated using the **validation dataset**.
 - First **decreases**, but after some time it may **start increasing**.
 - This increase indicates that the model is **overfitting**.

What is Overfitting?

- **Overfitting** occurs when the model:
- Learns training data **too perfectly**
- Memorizes noise and unnecessary details
- Performs **poorly on new unseen data**
- Example:
- A model trained on images of cats may memorize specific training images but fail to recognize **new cat images**.

Idea Behind Early Stopping

- Early stopping monitors the **validation error during training**.
- At first:
 1. Training error decreases
 2. Validation error also decreases
- After some time:
 1. Training error keeps decreasing
 2. But validation error starts increasing
- This means the model has started **overfitting**.
- So we **stop the training process at the point where validation error is minimum**.

Example Table

Epoch	Training Error	Validation Error
1	0.9	0.95
2	0.7	0.80
3	0.5	0.60
4	0.4	0.55
5	0.3	0.58
6	0.2	0.65

Observation:

- Validation error is **lowest at Epoch 4**
- After that it starts increasing

So the **best model is at Epoch 4**.

Early stopping will **stop training near Epoch 4**.

10. Parameter Typing and Parameter Sharing

- **Introduction**

- In deep learning models, there are many **parameters (weights and biases)** that the model learns during training. Sometimes the number of parameters becomes **very large**, which can cause:

1. Overfitting
2. Slow training
3. High memory usage

- To solve these problems, two important concepts are used:

1. **Parameter Typing**
2. **Parameter Sharing**

- Both techniques help make the **model simpler, faster, and more efficient.**

1. Parameter Typing

- **Meaning**
- **Parameter typing** means **grouping parameters according to their role or function in the model.**
- In a neural network, different parameters perform **different tasks**. Instead of treating all parameters the same, we classify them based on their **type**.
- **Example in Neural Networks**
- A neural network usually contains:
 1. **Weights**
 2. **Biases**
 3. **Filters (in CNN)**
- Each of these parameters has a **different purpose**.

Parameter Type

Purpose

Weights

Control the strength of connection between neurons

Bias

Helps adjust the output value

Filters

Detect patterns like edges or shapes in images

2. Parameter Sharing

- **Meaning**
- **Parameter sharing** means using the **same parameter values in multiple parts of the neural network**.
- Instead of learning separate weights for every connection, the **same weights are reused in many places**.
- This greatly **reduces the number of parameters** in the model.

Example in Convolutional Neural Networks (CNN)

- Parameter sharing is commonly used in **Convolutional Neural Networks (CNNs)**.
- In CNNs, a **filter (kernel)** moves across the entire image to detect patterns.
- Example:
- Suppose a filter detects **edges**.
- Instead of learning a new filter for every part of the image, the **same filter is used across the whole image**.

11. Sparse Representations

- Sparse representation means **most neuron outputs are zero**.
- Example:
 - [0, 0, 3.2, 0, 0, 1.1]
- Benefits:
 1. Efficient learning
 2. Better feature extraction
 3. Reduced overfitting
- Common in:
 1. Autoencoders
 2. Feature learning

12. Bagging (Bootstrap Aggregating)

- **Introduction**

- In machine learning, sometimes a **single model may not give very accurate predictions**. It may also suffer from **overfitting**, where the model performs well on training data but poorly on new data.
- To improve prediction accuracy, we use a technique called **Bagging**.
- **Bagging** stands for **Bootstrap Aggregating**.
- It is an **ensemble learning technique**, which means it **combines multiple models to produce a better final result**.
- Instead of using one model, bagging **trains many models and combines their predictions**.

Meaning of Bootstrap Aggregating

- Bagging has two important parts:
- **1. Bootstrap**
- Bootstrap means **creating multiple new datasets from the original dataset using random sampling.**
- In this process:
 1. Some data points may be repeated.
 2. Some data points may not appear in a sample.

- Example:
- Original dataset:
- A, B, C, D, E
- Bootstrap samples may look like:
- Sample 1 → A, B, C, A, D
Sample 2 → B, D, E, C, B
Sample 3 → A, C, E, E, D
- Each sample is **slightly different from the original data**

2. Aggregating

- Aggregating means **combining the predictions from multiple models** to get the final result.
- This is usually done by:
- **Voting** (for classification problems)
- **Averaging** (for regression problems)

Working of Bagging

- **Step 1: Create Multiple Bootstrap Datasets**
- From the original dataset, generate several **random samples with replacement**.
- Example:
- Original dataset → 1 dataset
- After bootstrapping → many datasets
- Dataset 1
Dataset 2
Dataset 3
Dataset 4

- **Step 2: Train Separate Models**

- A separate model is trained on **each dataset**.

- Example:

- Dataset 1 → Model 1

- Dataset 2 → Model 2

- Dataset 3 → Model 3

- Dataset 4 → Model 4

- Each model learns **different patterns** because the training data is different.

- **Step 3: Make Predictions**
- Each model gives its own **prediction**.
- Example:
 - Model 1 → Cat
 - Model 2 → Cat
 - Model 3 → Dog
 - Model 4 → Cat

- **Step 4: Combine Predictions**
- The final result is obtained by **combining all predictions**.
- For classification problems:
- **Majority voting**
- Example:
- Cat = 3 votes
Dog = 1 vote
- Final prediction = **Cat**
- For regression problems:
- **Average of predictions**
- Example:
- Model predictions = 20, 22, 21, 19
- Final prediction =
- $(20 + 22 + 21 + 19) / 4 = \mathbf{20.5}$

13. Ensemble Methods

- Ensemble methods combine **multiple models** to improve performance.
- Types:
 1. Bagging
 2. Boosting
 3. Stacking
- Idea:
 - Multiple weak models → Strong model
- 1. Benefits:
 2. Higher accuracy
 3. Robust predictions

14. Dropout

- In **deep learning**, neural networks contain **many neurons and connections (weights)**. When the network is trained on a dataset, sometimes it learns the **training data too well** and starts memorizing it instead of learning general patterns. This problem is called **overfitting**.
- To reduce overfitting, we use a regularization technique called **Dropout**.
- **Dropout is a technique where some neurons are randomly turned off (dropped) during training.**
- This helps the model learn **better and more general features**.

- Basic Idea of Dropout
- During training, some neurons in the neural network are **temporarily removed**.
- These neurons:
 - Do not participate in forward propagation
 - Do not update their weights during backpropagation

Before Dropout

All neurons are active.

nginx

Neuron1 → Neuron2 → Neuron3 → Neuron4

After Applying Dropout

Some neurons are randomly turned off.

nginx

Neuron1 → ✖ → Neuron3 → ✖

Here, Neuron2 and Neuron4 are **dropped temporarily**.

In the next training step, **different neurons may be dropped**.

How Dropout Works

- Dropout works in the following steps.
- **Step 1: Training Begins**
- The neural network starts learning from the training data.
- **step 2: Randomly Drop Neurons**
- A certain percentage of neurons are **randomly selected and deactivated**.
- Example:
- If a layer has 10 neurons and dropout rate = 0.5
- Then about **5 neurons will be dropped randomly**.

- **Step 3: Forward Propagation**

- Only the **remaining active neurons** pass information to the next layer.
- Dropped neurons **do not participate**.

- **Step 4: Backpropagation**

- Only active neurons update their **weights**.
- Dropped neurons **do not update weights**.

- **Step 5: Repeat for Next Iteration**
- In the next training step, **different neurons are dropped randomly.**
- So every iteration uses a **slightly different network structure.**

Dropout Rate

The **dropout rate** determines the percentage of neurons that will be dropped.

Example values:

Layer Type	Typical Dropout Rate
Input Layer	0.1 – 0.2
Hidden Layer	0.2 – 0.5

Common value used:

0.5 (50%)



This means **half of the neurons are randomly turned off during training.**

Why Dropout is Needed

- Sometimes neural networks develop **neuron dependency**.
- This means:
 - Some neurons depend heavily on certain other neurons to make predictions.
 - Example:
 - Neuron A always relies on Neuron B.
 - If Neuron B is missing or changes, the network performs poorly.
- Dropout prevents this by **randomly removing neurons**, forcing the network to learn **independent features**.